

PRODUCT MANAGEMENT · WORKFLOW

How the workflow runs

From a human roadmap input — reconciled against the live software and the backlog — into an orchestrated build pipeline, with three points where the human stays in control.

Schematic overview · automated pipeline with human decision gates · June 2026

The system turns direction into shipped work without me babysitting every step. Machines do the heavy, repeatable work — roadmap alignment, intake, analysis, building, testing, and our shared retro — while I keep authority over the few decisions that genuinely need judgment. Those decision points are the **gates**; everything between them runs autonomously. Because I write each request as a user story with explicit success criteria, the orchestrator can pick it up and run the moment I add it to the backlog and release it.

The front of the system is a **recurring product-management cadence**, not a one-off plan. On a regular rhythm my roadmap direction is re-aligned against what the software actually does today and against the existing backlog. Whenever I add a new item or re-align the roadmap, dependencies are identified continuously and a sequence is proposed; requests are merged where a shared implementation makes sense, overlaps are cleanly separated, and items that are too large are split. The critical path of user stories and bugs is re-identified rather than assumed.

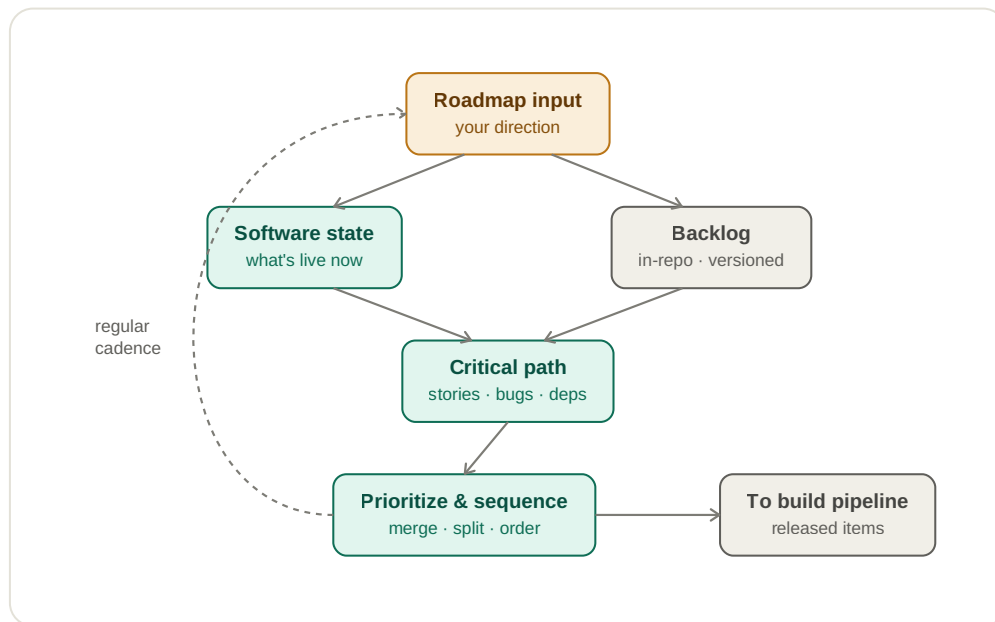
In my own setup, backlog and roadmap are managed directly in the repository, so they stay tightly interwoven with the rest of the system information. The team adaptation at the end still processes the work in the repo — it only pulls its input from Jira and writes results back to Jira for reporting. A retrospective after each item — which we run together — feeds improvements back into the backlog and updates the skills, so the agents work better in the next iteration. It is a compounding, Atomic-Habits-style effect: you can watch the system get a little better every time.

 You decide
(gate)  Automated
phase  Orchestrator  Shared
artifact

STAGE 1

Roadmap alignment & prioritization

The product-management cadence that runs before the build pipeline — repeated regularly.

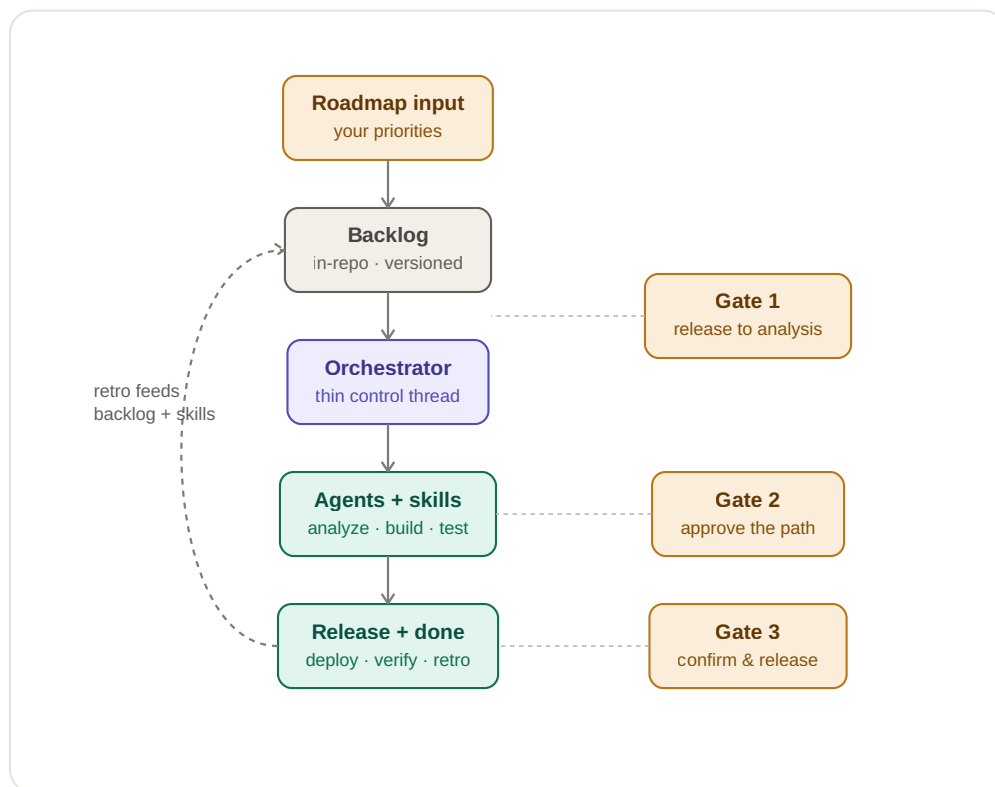


Roadmap is the only place I push direction in, and each request already arrives as a user story with success criteria. Every cycle it is reconciled against the current state of the software and the existing backlog: dependencies are identified, a sequence is proposed, related requests are merged where a shared implementation helps, overlaps are separated, and oversized items are split. Only the prioritized, released items then enter the build pipeline.

STAGE 2

The spine: roadmap to release

The high-level chain and where the human holds a gate.

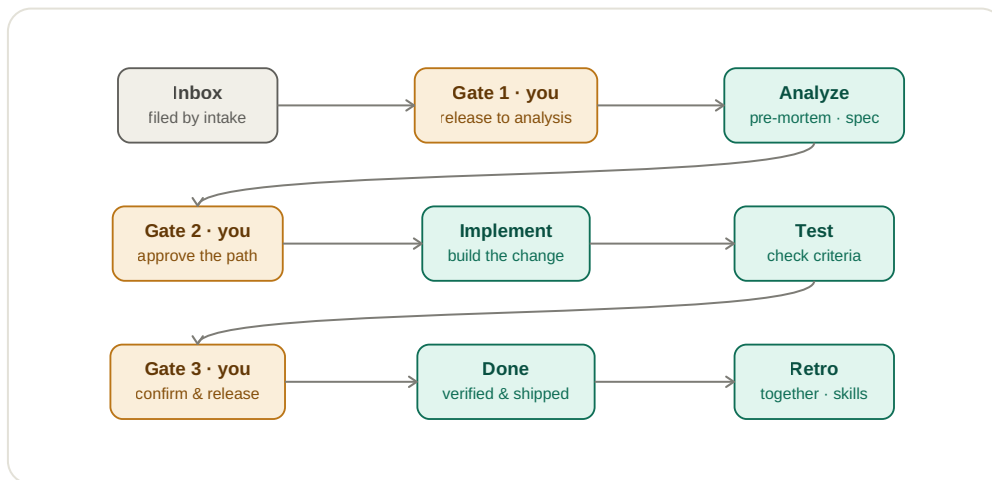


The orchestrator is a thin control thread: it picks the next released item and hands the work outward. The three amber gates are mine — release to analysis, approve the path, confirm test & release. Backlog and roadmap live in the repository, so they stay tightly coupled to the rest of the system, and a retrospective we run together feeds improvements back into the backlog and skills after each item.

STAGE 3

The engine: one item through its phases

What the orchestrator does with a single released item — and where my gates sit.



Each phase runs in its own sub-agent with isolated context; the orchestrator collects a compact result and moves the item one lane forward. Verification is done by a different agent than the one that wrote the change, and nothing in the inbox or explicitly excluded is ever touched. The retrospective is something we do together — I weigh in on what worked and what didn't — and its lessons update memory and the skills.

The three gates — my decisions

Gate 1 — Release to analysis. Intake files every new item into the inbox; only I move an item out of it. This is what turns a defined request into committed work and hands it to the orchestrator.

Gate 2 — Approve the path. Analysis returns several implementation options with a recommendation. I pick the path before any code is written.

Gate 3 — Confirm test & release. Where a check can't run fully automatically, I confirm it. Every deploy is manual and waits for my go.

Reading the whole loop

1. **Roadmap alignment (me + auto, recurring).** On a regular cadence I set direction. It is reconciled against the live software and the backlog; dependencies are identified, a sequence is proposed, and the critical path of user stories and bugs is re-identified — kept current, not fixed once.
2. **Intake (auto).** New items — which I write as user stories with explicit success criteria — are checked against the backlog: duplicates flagged, related requests merged where a shared implementation helps, overlaps separated, oversized items split, then filed in the inbox — never further.

3. **Gate 1 (me).** I release the items I want worked next; the orchestrator then takes over.
4. **Orchestrate + analyze (auto).** The orchestrator picks up a released item and runs analysis: a pre-mortem, acceptance criteria, and implementation options.
5. **Gate 2 (me).** I approve the recommended path.
6. **Implement & test (auto).** Separate agents build the change and check it against the acceptance criteria; verification is done by a different agent than the one that wrote the code.
7. **Gate 3 (me).** I confirm the result and authorize the release.
8. **Done & retro (me + auto).** The item closes and we run a short retrospective together — what worked, what didn't. It feeds improvements back into the backlog and updates the skills so the agents work better next time: a compounding, Atomic-Habits-style effect where the system gets a little better each iteration.

ADAPTATION

Adapting the model for a team on Jira

Same model, for an organization whose work already lives in Jira — with two changes and one addition.

The pipeline does not need a bespoke backlog. **Jira becomes the backlog feed:** the features and stories already in Jira flow in as the item stream, so the team keeps working where it already works. There is no separate roadmap tool — instead a short, plainly worded **Now / Next / Later** document (a single Word file) sets direction. Now is committed, Next is queued, Later is parked; clear enough that both people and agents can act on it. In short, Jira is the way in — the work itself is still processed in the repo, and Jira is updated afterwards as documentation (see below).

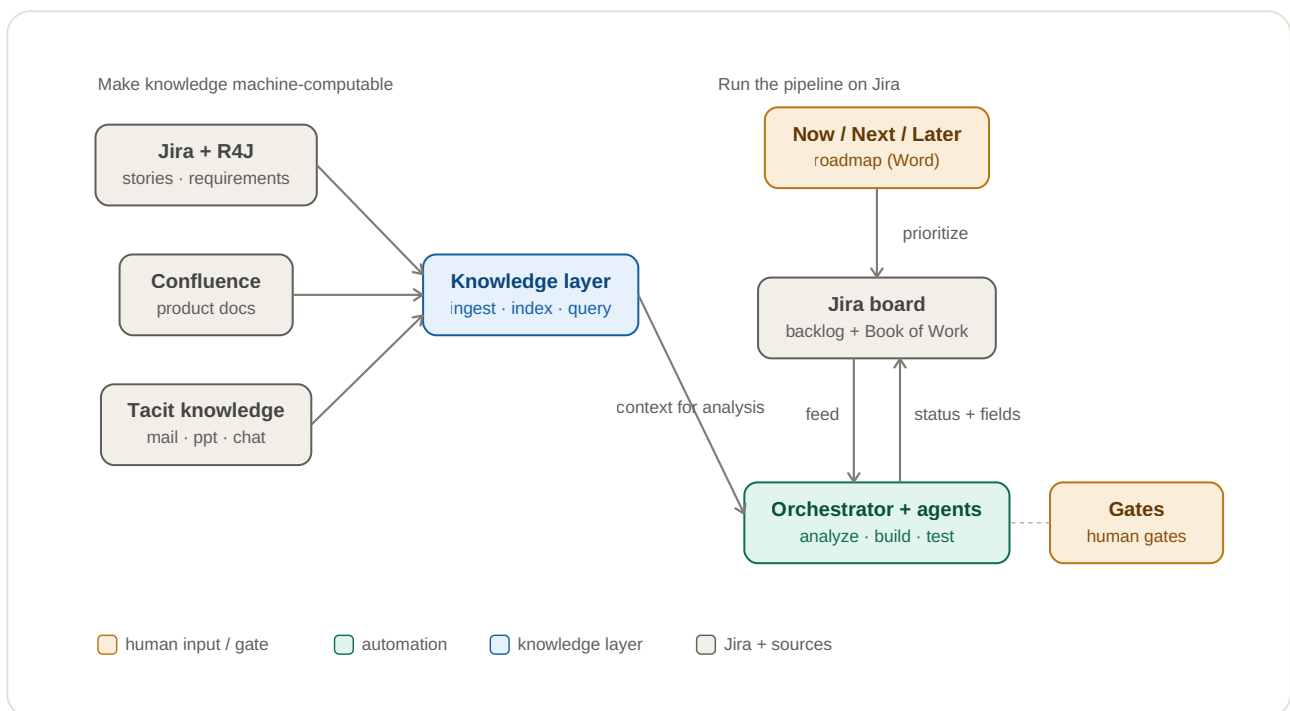
Analysis needs context that today is scattered: requirements in R4J, product information on Confluence, and a great deal of tacit knowledge in emails, slide decks and chats. The adaptation makes this knowledge **machine-computable:** each source is ingested into a knowledge layer that extracts structured facts and builds a searchable index, so an agent can retrieve the exact requirement, decision or constraint it needs while analyzing a request — instead of relying on a person to remember it. R4J already keeps requirements as Jira issues with built-in traceability to tests, so the knowledge layer's real job is the rest — the Confluence pages and tacit knowledge that Jira cannot see.

For the knowledge layer to be trustworthy, three things matter:

- **Freshness** — the index re-syncs when Confluence pages, Jira issues or documents change, so agents never reason on stale facts.
- **Access control** — the layer honours each source's permissions, so an agent never surfaces content the requester isn't allowed to see.
- **Provenance** — every retrieved fact carries a citation back to its source, so the analysis can be checked.

The loop is one way in, one way out. Jira feeds the work in; once everything is processed in the repo, an agent **writes the tickets' status and fields back to Jira** — purely as documentation for reporting and quarterly planning, so the company's Book of Work stays transparent there with no parallel tracker. These write-backs are bounded by Jira's own workflow and permission model.

Because a team ships shared code, the adaptation also adds a human **code-review gate**: the agent opens a pull request and a person reviews and approves the diff before it merges — a fourth gate alongside the three from the base model.



Left: scattered sources are made machine-computable in a knowledge layer the agents query during analysis. Right: the Now/Next/Later roadmap prioritizes the Jira-fed backlog; the orchestrator and its agents run the pipeline and write status and field updates back into Jira, keeping the Book of Work in one place. The human gates are unchanged.

Write-back guardrails

Enforced by Jira, not by trust. The project's workflow and permission scheme define which transitions are even possible; the agent cannot invent states or skip steps because Jira itself rejects them.

Native audit trail. Jira records every status and field change with author and timestamp in the issue history; the agent runs as a restricted service account, so its changes stay clearly attributed and scoped.

Human gates hold. Release, path approval and deploy still wait for a person; automation handles the movement in between.

TESTING

Testing: in-sprint automation from the acceptance criteria

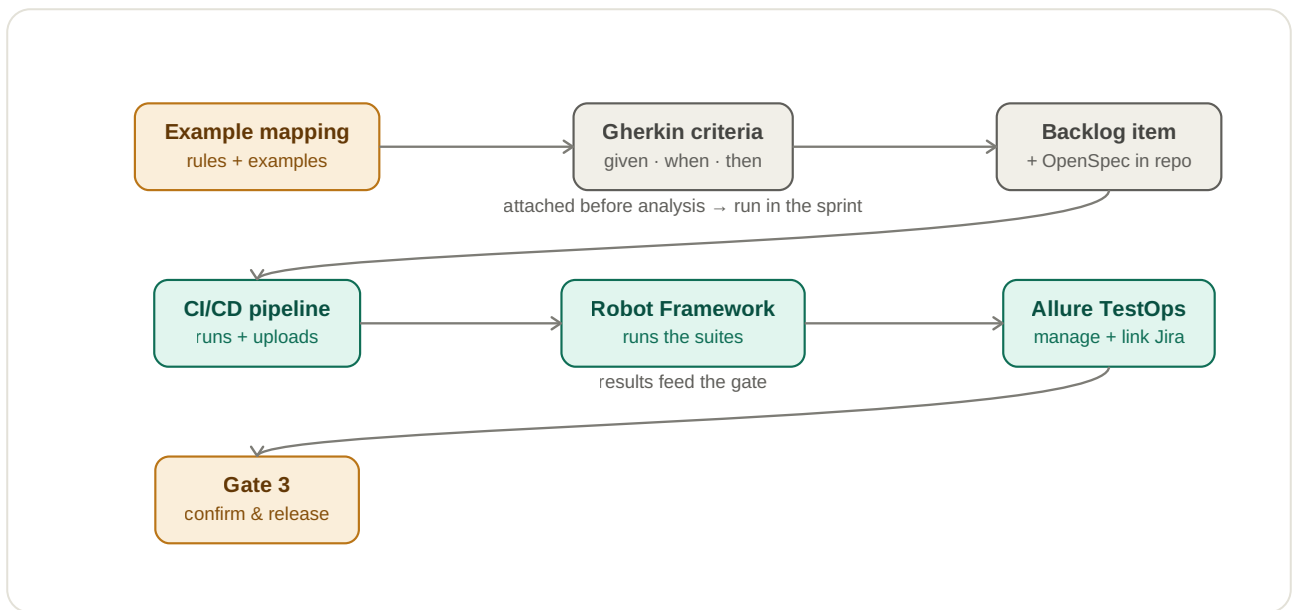
How the Gherkin acceptance criteria become executable tests and live monitoring.

The test contract is written first, not last. During **Example Mapping** — the very first step; in a team this is a three-amigos session of product, development and test, while in the base model I author the same criteria myself — we surface the rules and concrete examples for a story and capture the acceptance criteria in **Gherkin** style (given / when / then), then add them to the backlog item's details **before the orchestrator begins its analysis**. The same criteria are carried into the repository as an **OpenSpec** spec — the spec-driven format the coding agent consumes — so human and agent work against one shared, versioned definition of done through analysis, implementation and verification.

In the sprint those criteria become automated tests with **Robot Framework**. This is the testing step of the pipeline shown earlier, and on purpose it is handled by a different agent than the one that wrote the code — so no one marks their own homework. That agent turns the Gherkin criteria into Robot Framework tests and writes the small pieces of glue code (the step keywords) each line needs; the criteria text on its own is not yet runnable. Because this happens in the same sprint, a story is checked against its own acceptance criteria while it is still being built, not in a later phase.

Allure TestOps is the test-management and orchestration layer above the runs: it ingests the Robot Framework results, turns them into managed test cases and launches, and tracks pass/fail and flaky trends. Through its native, two-way **Jira integration** it links each test case to its Jira issue and surfaces the launch status on the ticket — so the test status of the Book of Work shows up in Jira without a separate push. The human Gate 3 (confirm test & release) reads from this rather than from ad-hoc checks.

The tests do not run on someone's laptop: a **CI/CD pipeline** runs the Robot Framework suites on every change and uploads the results to Allure TestOps automatically. End to end, one thread of **traceability** ties it together — roadmap item → Jira story → R4J requirement → Gherkin criterion → Robot Framework test → Allure TestOps result → Gate 3 sign-off — so any test result can be traced back to the requirement it proves.



Acceptance criteria are authored up front in Example Mapping, attached to the backlog item before analysis, and carried into the repo as an OpenSpec spec for the coding agent. In the sprint a CI/CD pipeline runs the Robot Framework suites generated from the criteria; Allure TestOps manages the runs and links them back to Jira; Gate 3 confirms before release.

Schematic overview of an automated build pipeline with human decision gates — shown for a single maintainer, adapted to a team on Jira, and extended with in-sprint test automation. Generic by design — implementation specifics omitted.